

说明

本文旨在系统性地阐述本公司基于 RISC-V 架构自主研发的芯片中所集成的一系列特色内核指令。内容将涵盖这些指令的微架构设计初衷、功能定义，并对其在特定算法场景下的性能增益与硬件开销进行量化分析，以全面评估其在提升处理器能效比方面的核心价值。

适用范围

适用范围	系列
通用 MCU	CH32

目录

说明	1
目录	2
表格索引	2
图片索引	2
第 1 章 扩展压缩指令	1
1.1 位域定义	1
第 2 章 MCPY 硬件搬运	2
2.1 简介	2
2.1.1 指令信息	2
2.2 性能测试	2
2.3 使用及注意事项	3
第 3 章 DLY 精确延时	5
3.1 简介	5
3.1.1 指令信息	5
3.1.2 相关 CSR 寄存器	5
3.2 特点	6
3.2.1 延时源配置	6
3.2.2 流水线控制	6
3.2.3 反馈机制	6
3.3 使用	6
历史版本	8
声明	9

表格索引

XW 扩展压缩指令一览	1
MCPY 指令位域定义	2
DLY 精确延时指令位域定义	5
DLY 配套 CSR 寄存器未定义	5
目前官方库可调用的接口	6

图片索引

与传统 memcpy 搬运对比	3
-----------------------	---

第1章 扩展压缩指令

RISC-V 标准的压缩指令扩展（C 扩展）主要针对字和双字（RV64C）的加载和存储操作进行了压缩。但在实际嵌入式开发中，对字节（Byte）和半字（Half-Word）的操作同样非常频繁。XW 扩展的出现就是为了填补这一空白。它新增了 8 条 16 位压缩指令，专门用于优化字节和半字的操作，有效减小了最终二进制文件的体积，这对于 Flash 和 SRAM 通常不大的 MCU 来说意义重大。

1.1 位域定义

表 1-1 XW 扩展压缩指令一览

RISC-V 标准	XW	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
c.flld/c.lq	c.lbu	001			uimm[0:4:3]				rs1'		uimm[2:1]			rd'			00
c.fldsp/c.lqsp	c.lhu	001			uimm[5:4:3]				rs1'		uimm[2:1]			rd'			10
c.fsd/c.sq	c.sb	101			uimm[0:4:3]				rs1'		uimm[2:1]			rs2'			00
c.fsdsp/c.sqsp	c.sh	101			uimm[5:4:3]				rs1'		uimm[2:1]			rs2'			10
c.lw	c.lw	010			uimm[5:3]				rs1'		uimm[2:6]			rd'			00
c.sw	c.sw	110			uimm[5:3]				rs1'		uimm[2:6]			rs2'			00
Reserved	c.lbusp	100			00			uimm[3:0]			00			rd'			00
Reserved	c.lhusp	100			00			uimm[3:1:4]			01			rd'			00
Reserved	c.sbsp	100			00			uimm[3:0]			10			rs2'			00
Reserved	c.shsp	100			00			uimm[3:1:4]			11			rs2'			00
c.lwsp	c.lwsp	010			uimm[5]			rd != 0			uimm[4:2:7:6]						10
c.swsp	c.swsp	110			uimm[5:2:7:6]						rs2						10

本系列内核自定义实现了 8 条压缩读写指令，功能参考 c.lw/c.sw/c.lwsp/c.swsp 指令对半字和字节宽度的数据操作进行了补充，其中 c.lbu/c.lhu/c.sb/c.sh 指令占用了双精度浮点压缩指令的编码空间，冲突情况如上表所示。

XW 自定义扩展压缩指令的使用条件

XW 扩展是 WCH 基于 RISC-V 标准指令集架构自定义的扩展指令集，其中的压缩指令需在特定工具链环境下方可正常编译与使用。具体要求如下：

- 工具链要求：**必须使用 WCH 官方提供的 RISC-V 工具链。标准开源 RISC-V 工具链（如 GNU 官方发行版）默认不包含 XW 扩展的支持，无法识别或生成相关压缩指令；
- 编译参数要求：**在编译时，需在 -march 参数中显式添加 xw 扩展标识，以通知编译器启用 XW 扩展支持。

第2章 MCPY 硬件搬运

1.2 简介

该指令实现从源地址到目标地址的连续内存字节拷贝。其中，rs1 指定拷贝结束地址（不包含该地址），rs2 指定拷贝起始地址，rd 指定目标起始地址。指令执行期间，硬件自动从源起始地址逐字节读取数据，并写入至目标起始地址，直至源地址累加至结束地址为止。用于替代 memcpy 函数，实现连续的内存搬运功能，指令寄存器中保存起始地址，目标地址，结束地址的地址信息，所有地址均无对齐要求。

1.2.1 指令信息

位域定义（Format）：

表 2-1 MCPY 指令位域定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs3					00		rs2					rs1					111			00000					MISC_MEM						
Name					Bit					Function																					
rs3					31:27					结束地址寄存器编码																					
RSV					26:25					固定值 00																					
rs2					24:20					起始地址寄存器编码																					
rs1					19:15					目标地址寄存器编码																					
fun3					14:12					固定值 111																					
fun5					11:7					固定值 00000																					
fun7					6:0					固定值 0001111																					

汇编语法（Syntax）：

```
mcpy rs1, rs2, rs3
```

执行逻辑（Pseudocode）

```
while (rs2 < rs1) {
    mem[rs3] = mem[rs2];
    rs2 = rs2 + 1;
    rs3 = rs3 + 1;
}
```

中断响应：

该指令为可中断指令。在 mcpy 执行过程中，若检测到有效中断请求，硬件将在当前字节传输完成后（或下一个指令周期边界处）暂缓执行。中断返回后，指令继续执行，拷贝流程与未发生中断时的最终结果一致。该机制保证了大块数据传输场景下系统的实时响应能力。

1.3 性能测试

性能评估方法

为量化不同数据搬运方式在非特权模式下的执行效率，本测试利用 RISC-V 内核的 ucycle 寄存器（地址 0xC00）进行周期精确计数。优化等级-OFast，以保证库函数性能最高。

测试流程如下：

1. 通过内嵌汇编指令 csrr 读取当前机器周期数，并记录为起始时间戳 starttime。
2. 调用待测数据搬运函数 CopyMethod(Adapter, mydata, range)。
3. 搬运完成后，再次读取 ucycle 寄存器，获取结束时间戳 stoptime。

测试代码如下：

```
__asm volatile("csrr %0,"
               "0xC00"
               : "=r"(starttime))
```

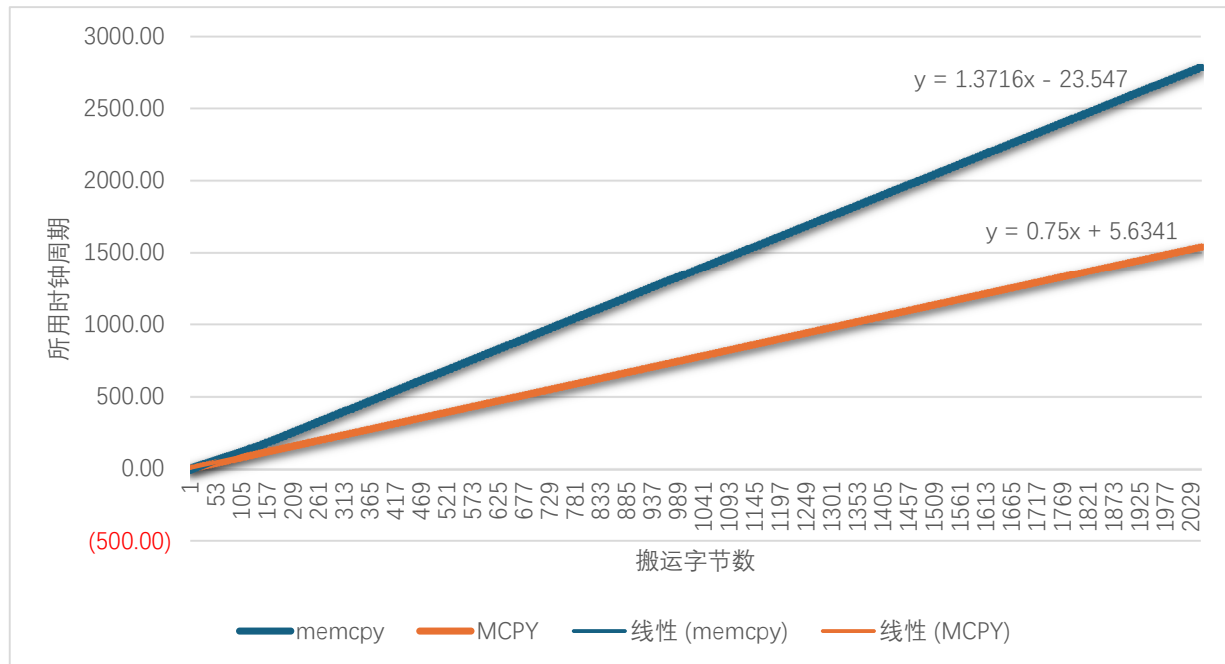
```

:);
CopyMethod(Adapter, mydata, range);
__asm volatile("csrr %0,"
               "0xC00"
               : "=r"(stoptime)
               :);

```

本测试在 range = 1 ~ 2048 字节范围内，逐点统计了两种数据搬运方法的时钟周期消耗，并据此绘制性能曲线（如下图所示）。实验结果表明，在整个测试区间内，两种搬运方式的时钟周期开销均与搬运数据量呈近似线性正相关关系。

图 2-1 与传统 memcpy 搬运对比



对实测数据进行线性拟合，得到两种方法的拟合表达式分别为：

$$t_{memcpy} = 1.3716x - 23.547$$

$$t_{MCPY} = 0.75x + 5.6341$$

由拟合结果及曲线斜率对比可见，MCPY 方法在单字节搬运效率上显著优于常规 memcpy 方法。尤其当搬运数据量较大时，MCPY 的累积周期节省更为可观，能够有效降低 CPU 在数据搬移任务中的占用时间，提升系统整体吞吐率。

1.4 使用及注意事项

调用方式：

MCPY 指令的汇编语法详见本章第 1 节。在官方软件开发库（SDK）环境下，用户可直接调用封装接口 ASM_MCPY() 来使用该指令。

注意事项：

1. 地址对齐与重叠限制

MCPY 指令执行从低地址向高地址方向的顺序数据搬运。严禁在源数据区的尾端与目标数据区的首端存在地址重叠时使用本指令。若发生此类重叠，将导致源数据在被读取前被目标写操作覆盖，进而引发搬运结果错误。

2. 中断响应行为

中断：MCPY 指令执行期间允许响应中断。中断服务程序执行完毕后，指令将自动恢复继续执行，无需软件额外干预。

调试请求（含单步）：调试请求或单步请求将等待当前 MCPY 指令完整执行结束后再予以响应，不会打断搬运过程。

3. 异常处理

若 MCPY 指令的源地址或目标地址触发以下保护机制，将产生不可恢复（不可恢复）异常，系统需进行上层复位或错误处理：

- 物理内存保护（PMP）访问权限限制；
- 硬件断点（Hardware Breakpoint）命中。

4. 寄存器副作用

MCPY 指令执行期间，源地址寄存器（rs2）和目标地址寄存器（rs3）会随搬运进度递增更新。指令执行完成后，rs2 与 rs3 的最终值取决于搬运终止时的地址位置，不可预测，用户不应依赖其后续值进行编程。

第2章 DLY 精确延时

2.1 简介

自定义延时指令是一条以单指令开销为代价的高精度延时指令。该指令能够实现精确至单个时钟周期级别的延时控制，有效规避了传统定时器或软件循环延时中因指令跳转、流水线冲刷及分支预测失败等带来的时序不确定性问题。

2.1.1 指令信息

表 2-1 DLY 精确延时指令位域定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
imm											rs1					func3			match		div	sel	func7								
Name		Bit		Function																											
imm		31:20		延时立即数，根据配置可表示延时的时钟周期数或分频后的周期数。																											
rs1		19:15		匹配的 rs1 寄存器编码。																											
func3		14:12		001b。																											
match		11:9		匹配的指令类型，在延时未结束时匹配的指令类型无法执行 000: 保留; 001: 匹配 load 指令; 010: 匹配 store 指令; 011: 匹配 load/store 指令; 100: 匹配 delay 指令; 101: 保留; 110: 匹配所有指令（流水线暂停）; 111: 匹配指定 func3 和 opcode 的指令（匹配值由 csr 寄存器配置）																											
div		8		使用主频时钟或分频时钟计数 1: 使用分频周期计数（分频系数由 csr 寄存器配置）; 0: 使用主频周期计数。																											
sel		7		延时数选择: 1: rs1 + rs2; 0: rs1 + imm。																											
fun7		6:0		0001011b																											

2.1.2 相关 CSR 寄存器

该 CSR 寄存器专为 DLY 自定义延时指令设计，是延时功能的核心控制与状态接口。其主要功能包括：

- 读操作：读取当前延时计数器的运行状态（如计数值、溢出标志等），供软件查询延时进度或判断延时是否完成；
 - 写操作：配置延时参数（如延时周期数、时钟分频系数等），为 DLY 指令的执行提供所需配置。
- 延时指令控制寄存器（U_NONS_DLY_0：0x8C0）：

表 2-2 DLY 配套 CSR 寄存器未定义

Name	Bit	Function
dly_mask	31:24	dly 指令 match=111 时匹配的指令参数掩膜。
dly_data	23:16	dly 指令 match=111 时匹配的 {func, opcode}。
CPU_dly_busy_sta	15	内核处于延时状态标志。
dly_tout	14	超时信息，在延时范围内无匹配的指令

Name	Bit	Function
		请求。
dly_ie	13	延时中断请求，使能后如果产生超时情况，即产生一个软件中断请求。。
dly_priv	12	1：机器模式和用户模式均允许匹配 0：在用户模式下执行的 delay 操作不匹配机器模式中的指令。
Reserved	11:10	保留。
dly_freq	9:0	延时计数分频系数配置，若配置为主频时钟频率，可实现 us 为单位延时。

2.2 特点

2.2.1 延时源配置

双时钟源选择：通过 div 位选择使用主频时钟或分频时钟。

动态延时计算：通过 sel 位选择延时值是 rs1 + imm（立即数）还是 rs1 + rs2（寄存器值），允许在运行时动态调整延时长度。

分频配置：通过 CSR 寄存器 U_NONS_DLY_0 中的 dly_freq 字段配置分频系数，甚至可以配置为以亚微秒（us）为单位的延时。

2.2.2 流水线控制

通过 match 位域，这条指令可以决定在延时期间“阻塞谁”：

- 精准阻塞：可以只阻塞 Load、Store 或 Load/Store 指令。
- 全阻塞模式：match=110 时，阻塞所有指令，相当于完全暂停流水线。
- 智能阻塞：match=111 时，由 CSR 寄存器配置匹配特定的 func3 和 opcode 指令，实现对特定功能指令的拦截。

2.2.3 反馈机制

通过 U_NONS_DLY_0 寄存器提供状态监控：

- 状态标志：CPU_dly_busy_sta 指示内核是否处于延时状态。
- 超时检测：dly_tout 记录在延时期间是否有未预期的指令请求（即本不该发生的指令发生了）。
- 中断机制：dly_ie 使能后，若发生超时，可触发软件中断，用于调试或异常处理。

2.3 使用

当前版本的工具链尚未集成 DLY 自定义延时指令的汇编器支持。为便于用户在现有开发环境中使用该指令功能，官方软件库（SDK）提供了以下一组内联函数（Intrinsic）作为替代调用接口：

表 2-3 目前官方库可调用的接口

函数接口	功能描述
DelayIntrinsic()	延时功能主入口，执行指定周期的精确延时
__set_DLY(value)	配置延时周期寄存器（写入延时计数值）
__get_DLY()	读取当前延时计数器的计数值
__clear_DLY_OV()	清除延时溢出（Overflow）状态标志

简单示例：

```
__asm("nop");

__set_DLY(0, 0, ENABLE, ENABLE, 10);

DelayIntrinsic(DLY_SRC_HCLK, DLY_Match_Load_Store, 0, 10);
```



```

__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");

__asm("lw x0, 0x00(x0)");

```

1. 计时启动：

调用 DelayIntrinsic 后，DLY 模块立即开始对指定窗口内的指令进行监控并计时。

2. 指令匹配结果：

● 匹配成功（在指定周期内检测到 Load/Store 指令）：

DLY 模块将立即挂起 CPU 执行，进入精确延时状态。待设定的延时周期数到达后，自动解除挂起，CPU 从匹配指令的下一条指令处恢复执行。

● 匹配失败（在指定周期内未检测到任何 Load/Store 指令）：

触发软件中断，通知软件未在预期窗口内捕获到目标指令。此时不会进入延时挂起状态，需由中断服务程序进行相应处理。

3. 周期延时用法（匹配指令自身）：

若应用场景需要实现固定周期延时，可将匹配类型配置为 DLY_Match_Delay，使 DLY 指令匹配自身。此时无需额外目标指令，调用后 DLY 模块直接挂起 CPU，延时指定周期后自动恢复。如下所示：

```

while (1)
{
    /* Perform intrinsic delay operation */
    DelayIntrinsic(DLY_SRC_HCLK, DLY_Match_Delay, 0, 100);
    /* Toggle GPIOA pin 1 output state */
    GPIOA->OUTDR ^= GPIO_Pin_1;
}

```

历史版本

日期	版本	变更内容
2026/8/6	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。